DATA SCIENCE - 22DS102006

7. Data Aggregation

a. Create a tabular dataset as a DataFrame and split data into groups using groupby method including single key and multiple key values. Select group by considering single and multiple columns.
b. Compute summary statistics such as sum, mean and standard deviation for the grouped data using aggregate method.
c. Use groupby function to split data into groups based on one column, miultiple columns, compute summary statistics and perform exploratory data analysis. Consider any online dataset for processing.

Create a tabular dataset as a DataFrame and split data into groups using groupby method including single key and multiple key values. Select group by considering single and multiple columns.

```
import pandas as pd

# Create a tabular dataset
data = {
   'Name': ['Ravi', 'Sita', 'Arun', 'Priya', 'Kiran', 'Anil', 'Lakshmi', 'Rahul'],
   'Department': ['CSE', 'CSE', 'ECE', 'ECE', 'CSE', 'ECE', 'CSE', 'ECE'],
   'Year': [2, 2, 3, 3, 2, 3, 3, 2],
   'Marks': [85, 90, 78, 88, 75, 82, 95, 70]
}

# Create DataFrame
df = pd.DataFrame(data)

print("Original DataFrame:")
print(df)

*/ # 1. Group by a SINGLE KEY  */

print("\n1. Group by Department:")

group1 = df.groupby('Department')

for name, group in group1:
   print("\nDepartment:", name)
```

```
    print(group)

# Calculate average marks for each department
print("\nAverage Marks by Department:")
print(df.groupby('Department')['Marks'].mean())

*/ # 2. Group by MULTIPLE KEYS */

print("\n2. Group by Department and Year:")

group2 = df.groupby(['Department', 'Year'])

for name, group in group2:
    print("\nGroup:", name)
    print(group)

# Calculate average marks for each Department and Year
print("\nAverage Marks by Department and Year:")
print(df.groupby(['Department', 'Year'])['Marks'].mean())
```

Output:

Original DataFrame:

	Name	Department	Year	Marks
0	Ravi	CSE	2	85
1	Sita	CSE	2	90
2	Arun	ECE	3	78
3	Priya	ECE	3	88
4	Kiran	CSE	2	75
5	Anil	ECE	3	82
6	Lakshmi	CSE	3	95
7	Rahul	ECE	2	70

1. Group by Department:

Department: CSE

	Name	Department	Year	Marks
0	Ravi	CSE	2	85
1	Sita	CSE	2	90
4	Kiran	CSE	2	75
6	Lakshmi	CSE	3	95

Department: ECE

	Name	Department	Year	Marks
2	Arun	ECE	3	78
3	Priya	ECE	3	88
5	Anil	ECE	3	82
7	Rahul	ECE	2	70

Average Marks by Department:

Department

CSE 86.25

```
ECE    79.50
Name: Marks, dtype: float64
```

2. Group by Department and Year:

```
Group: ('CSE', 2)
   Name Department  Year  Marks
0   Ravi        CSE     2     85
1   Sita        CSE     2     90
4  Kiran        CSE     2     75

Group: ('CSE', 3)
     Name Department  Year  Marks
6  Lakshmi        CSE     3     95

Group: ('ECE', 2)
   Name Department  Year  Marks
7  Rahul        ECE     2     70

Group: ('ECE', 3)
   Name Department  Year  Marks
2   Arun        ECE     3     78
3  Priya        ECE     3     88
5   Anil        ECE     3     82
```

Average Marks by Department and Year:

```
Department  Year
CSE         2       83.333333
            3       95.000000
ECE         2       70.000000
            3       82.666667
```

Name: Marks, dtype: float64

Compute summary statistics such as sum, mean and standard deviation for the grouped data using aggregate method.

```
import pandas as pd

# Create a DataFrame
data = {
   'Department': ['CSE', 'CSE', 'CSE', 'ECE', 'ECE', 'ECE',
                  'EEE', 'EEE', 'EEE'],
   'Marks': [80, 75, 85, 70, 65, 75, 90, 85, 95],
   'Attendance': [90, 85, 95, 80, 75, 85, 95, 90, 100]
}

df = pd.DataFrame(data)

print("Original DataFrame:")
print(df)

# Group data by Department and calculate summary statistics
result = df.groupby('Department').aggregate({
   'Marks': ['sum', 'mean', 'std'],
   'Attendance': ['sum', 'mean', 'std']
})

print("\nSummary Statistics:")
print(result)
```

Output:

```
Original DataFrame:
  Department  Marks  Attendance
0        CSE     80          90
1        CSE     75          85
2        CSE     85          95
3        ECE     70          80
4        ECE     65          75
5        ECE     75          85
6        EEE     90          95
7        EEE     85          90
8        EEE     95         100
```

Summary Statistics:

	Marks			Attendance		
	sum	mean	std	sum	mean	std
Department						
CSE	240	80.0	5.0	270	90.000000	5.000000
ECE	210	70.0	5.0	240	80.000000	5.000000
EEE	270	90.0	5.0	285	95.000000	5.000000

Use groupby function to split data into groups based on one column, miultiple columns, compute summary statistics and perform exploratory data analysis. Consider any online dataset for processing.

```
import pandas as pd
# Read the online Iris dataset
url="https://raw.githubusercontent.com/mwaskom/seaborn-data/master/iris.csv"
df = pd.read_csv(url)

# Display first 5 records
print("First 5 Records:")
print(df.head())

# Display dataset information
print("\nDataset Information:")
print(df.info())

# Display statistical summary
print("\nStatistical Summary:")
print(df.describe())

# 1. GroupBy using a single column
print("\n1. GroupBy using a Single Column:")
group_single = df.groupby('species')['sepal_length'].agg(
   ['count', 'sum', 'mean', 'std', 'min', 'max']
)
print(group_single)

# 2. GroupBy using multiple columns

print("\n2. GroupBy using Multiple Columns:")
group_multiple = df.groupby(
   ['species', 'sepal_width']
)['sepal_length'].mean()
print(group_multiple.head(15))

# 3. Summary statistics for each species

print("\n3. Summary Statistics:")
summary = df.groupby('species').agg({
   'sepal_length': ['mean', 'std', 'min', 'max'],
   'sepal_width': ['mean', 'std', 'min', 'max'],
```

```
    'petal_length': ['mean', 'std', 'min', 'max'],
    'petal_width': ['mean', 'std', 'min', 'max']
})

print(summary)

# 4. Exploratory Data Analysis

print("\n4. Number of Records in Each Species:")
print(df['species'].value_counts())

print("\nAverage Petal Length by Species:")
print(df.groupby('species')['petal_length'].mean())

print("\nAverage Petal Width by Species:")
print(df.groupby('species')['petal_width'].mean())

print("\nMaximum Sepal Length by Species:")
print(df.groupby('species')['sepal_length'].max())
```

Output:

Sample Output

First 5 Records:

	sepal_length	sepal_width	petal_length	petal_width	species
0	5.1	3.5	1.4	0.2	setosa
1	4.9	3.0	1.4	0.2	setosa
2	4.7	3.2	1.3	0.2	setosa
3	4.6	3.1	1.5	0.2	setosa
4	5.0	3.6	1.4	0.2	setosa

1. GroupBy Using a Single Column

species	count	sum	mean	std	min	max
setosa	50	250.3	5.006	0.352490	4.3	5.8
versicolor	50	296.8	5.936	0.516171	4.9	7.0
virginica	50	329.4	6.588	0.635880	4.9	7.9

2. GroupBy Using Multiple Columns

```
species     sepal_width
setosa      2.3          5.00
            2.4          5.05
            2.5          5.12
            2.6          5.00
            2.7          5.10
            2.8          5.20
            2.9          5.30
            3.0          4.90
            3.1          4.90
            3.2          4.70
...
```

3. Summary Statistics

```
          sepal_length                sepal_width
               mean      std  min  max       mean      std
species
setosa          5.006  0.352490  4.3  5.8      3.428    0.379
versicolor      5.936  0.516171  4.9  7.0      2.770    0.314
virginica       6.588  0.635880  4.9  7.9      2.974    0.322
```

4. Exploratory Data Analysis

Number of Records in Each Species:

```
setosa       50
versicolor   50
virginica    50
Name: count, dtype: int64
```

Average Petal Length by Species:

```
setosa        1.462
versicolor    4.260
virginica     5.552
Name: petal_length, dtype: float64
```

Average Petal Width by Species:

```
setosa        0.246
versicolor    1.326
virginica     2.026
Name: petal_width, dtype: float64
```

Maximum Sepal Length by Species:

```
setosa       5.8
versicolor   7.0
virginica    7.9
Name: sepal_length, dtype: float64
```

7. Data Aggregation

a. GroupBy and Aggregation

```
df = pd.DataFrame({'Category': ['A', 'A', 'B', 'B'],
                   'Data': [10, 20, 30, 40]})
grouped = df.groupby('Category')
print(grouped['Data'].sum())
```

b. Multiple Aggregates

```
print(grouped['Data'].agg(['mean', 'std']))
```